Product Requirements Document (PRD)

CarCraft — Car Dealership Inventory & Purchase Management System

Prepared for: Incubyte Software Craftsperson Internship Assessment	**Document Version:** 1.0 (Proposed — Ready for Implementation)
Product Type: Full-Stack Single-Page Web Application (SPA)	**Target Delivery:** 25 October 2026, 10:00 PM IST
Backend & Database: Python + FastAPI + SQLite (Persistent Storage)	**Frontend Stack:** React + HTML5 + CSS3 + Tailwind CSS
Testing Suite: pytest + React Testing Tools	**Methodology:** Test-Driven Development (TDD: Red-Green-Refactor)

1. Executive Summary

CarCraft is a web-based car dealership inventory and purchase management application that allows customers to discover available vehicles, search and filter inventory, purchase vehicles, and review their purchase history. Dealership administrators can manage vehicle listings, monitor stock, restock vehicles, and maintain accurate inventory records.

The product is designed around a fundamental engineering principle: **Every important action must have predictable behaviour, appropriate authorization, and tests that verify its correctness.**

CarCraft is being built for the Incubyte Software Craftsperson Internship assessment. Therefore, product quality includes both the application's functionality and the engineering practices used to deliver it. The project prioritizes:

1. **Test-Driven Development (TDD)** with a genuine Red-Green-Refactor workflow.
2. **Secure authentication and authorization** enforced on the server side.
3. **Reliable inventory and purchase operations** with atomic database transactions.
4. **Clean, maintainable code** with deliberate, pragmatic design choices.
5. **Transparent and responsible AI-assisted development** with complete documentation.
6. **A polished, responsive user experience** built with React and Tailwind CSS.

The objective is not to maximize the number of features. It is to deliver a coherent, usable, tested product with clear reasoning behind its design decisions.

2. Problem Statement

2.1 The Problem

Car dealerships need a reliable way to maintain vehicle inventory and make available vehicles discoverable to customers. A poorly designed inventory application introduces several critical risks:

- Customers cannot easily find vehicles matching their preferences.
- Vehicle availability becomes inaccurate or negative after concurrent or invalid purchases.
- Invalid requests produce inconsistent inventory records.
- Unauthorized users modify or delete inventory due to weak backend access control.
- Administrators struggle to identify which vehicles need restocking.
- Customers have no way to review their previous purchases.
- Developers introduce regressions when modifying existing functionality without automated tests.

These problems are not exclusively visual or operational; they require careful backend design, validation, database management, security, and automated testing.

2.2 Our Proposed Solution

CarCraft provides a single-page application (SPA) backed by a RESTful API and a persistent relational database. Customers can register, log in, browse inventory, search for vehicles, purchase available stock, and view their purchase history. Administrators can manage vehicle listings and inventory quantities. The backend enforces business rules regardless of what the frontend displays or allows, and automated tests verify authorization, input validation, and inventory consistency.

2.3 Product Vision

Make discovering and managing dealership inventory simple for users, while making correctness and maintainability fundamental to the software itself.

3. Product Goals and Non-Goals

3.1 Product Goals

ID	Goal	Expected Outcome
G1	Provide reliable inventory management	Vehicle information and stock remain consistent across restarts and operations.
G2	Make vehicles easy to discover	Customers can search and filter inventory by make, model, category, and price.
G3	Support safe purchases	Invalid purchases are rejected without corrupting stock or overselling inventory.

ID	Goal	Expected Outcome
G4	Enforce access control	Administrative operations strictly require administrator permissions on the server.
G5	Provide purchase traceability	Customers can view their recorded purchases with preserved historical unit prices.
G6	Demonstrate software craftsmanship	The project has meaningful tests, readable code, and a credible TDD development history.
G7	Deliver a polished user experience	The application is responsive, accessible, and easy to navigate on desktop and mobile.
G8	Demonstrate responsible AI usage	AI assistance is documented transparently and every submitted implementation is verified.

3.2 Non-Goals

The initial release will not attempt to become a complete commercial dealership platform. The following are explicitly not required for the MVP because they add complexity without materially improving assessment readiness:

- Real payment processing, financing, and loan applications.
- Vehicle delivery management and insurance integration.
- Dealer-to-dealer transactions and multiple dealership branches.
- Production-scale distributed infrastructure or advanced recommendation systems.
- Native mobile applications.

4. Target Users

Both customers and inventory administrators are supported as primary user groups, alongside the technical reviewer evaluating the submission.

4.1 Customer

Profile: A person who wants to browse available cars and record purchases through the application.

- **Needs:** Discover available vehicles; search by make and model; filter by category and price; view accurate prices and stock availability; purchase a vehicle when stock is available; review previous purchases; receive clear explanations when an operation fails.
- **Success Criteria:** The customer can find a suitable vehicle, complete a valid inventory purchase, and see the resulting stock update and purchase history without confusion.

4.2 Inventory Administrator

Profile: A dealership employee responsible for maintaining vehicle listings and stock.

- **Needs:** Add new vehicle listings; correct vehicle information; update prices and categories; delete listings when appropriate; restock vehicles; inspect current stock levels; prevent unauthorized users from changing inventory.
- **Success Criteria:** Inventory changes are straightforward, accurate, and protected by strict backend role permissions.

4.3 Developer or Technical Reviewer

Profile: The Incubyte assessor reviewing the engineering quality, test suite, and authenticity of the submission.

- **Needs:** Understand the application architecture; run the project locally using documented instructions; execute the automated test suite; inspect meaningful Git commits; understand the rationale behind design choices; review the mandatory AI usage documentation (`README.md` and `PROMPTS.md`).
- **Success Criteria:** The reviewer can run the application and tests without guesswork and evaluate the implementation efficiently.

5. User Roles and Permissions

The application enforces two distinct roles: `CUSTOMER` and `ADMIN`. Permissions are enforced server-side on every protected endpoint. For the MVP, the initial administrator account is created through a controlled setup mechanism rather than allowing public registration with an administrator role.

Action	Customer	Admin
Register an account	Yes	Created through controlled setup process
Log in	Yes	Yes
View vehicle inventory	Yes	Yes
Search and filter vehicles	Yes	Yes
Purchase available stock	Yes	Yes (if enabled by policy)
View own purchase history	Yes	Yes (own purchases)
Add a vehicle	No	Yes
Update a vehicle	No	Yes
Delete a vehicle	No	Yes
Restock a vehicle	No	Yes
View another customer's purchase history	No	No (unless separately justified)

6. User Stories and Acceptance Criteria

Epic A: Authentication

US-01: Register an Account

As a customer, I want to register an account so that I can access protected functionality.

- The user supplies required registration details; email addresses must be valid and unique.
- Passwords must satisfy the configured password policy and are stored as secure hashes, never as plaintext.
- Invalid registration requests return an appropriate error; successful registration persists the account in the database.
- Public registration cannot assign the administrator role.

US-02: Log In

As a registered user, I want to log in so that I can access my account.

- Valid credentials produce a JWT access token; invalid credentials are rejected.
- The token contains the minimum necessary identity and authorization claims.
- Expired or invalid tokens cannot authorize protected operations, and passwords are never returned in API responses.

US-03: Access Protected Functionality

As a registered user, I want the system to protect operations requiring authentication.

- Protected endpoints reject requests without valid authentication, and administrator endpoints reject customers.
- Authentication failures return consistent responses, with authorization verified on every protected request.

Epic B: Vehicle Discovery

US-04: View Vehicle Inventory

As a customer, I want to view available vehicles so that I can explore the dealership's stock.

- The API returns vehicle records from persistent storage including ID, make, model, category, price, and stock quantity.
- Unavailable vehicles are not presented as purchasable; an empty inventory produces a useful empty state; the frontend displays loading and error states.

US-05: Search and Filter Vehicles

As a customer, I want to find vehicles matching my preferences.

- Search supports make and model; category filtering and minimum/maximum price filters are supported and work together in combination.

- Invalid price ranges are rejected or handled according to a documented policy; no matching results produce a helpful message.

Epic C: Inventory Management

US-06: Add a Vehicle

As an administrator, I want to add vehicles to inventory.

- All required fields are validated: prices must be positive and stock quantities must be non-negative integers.
- New vehicles receive unique IDs, unauthorized requests are rejected, and successful creation persists the vehicle.

US-07: Update a Vehicle

As an administrator, I want to update vehicle details when information changes.

- Only an administrator can perform the update, and the target vehicle must exist.
- Required fields and values are validated; invalid updates do not leave partially modified data; existing purchase records remain valid if vehicle details change.

US-08: Delete a Vehicle

As an administrator, I want to remove a vehicle listing when necessary.

- Only an administrator can delete a vehicle; requests for nonexistent vehicles return 404 Not Found.
- Deletion does not erase historical purchase records. For the MVP, deleting a vehicle referenced by existing purchase records is prevented with a clear 409 Conflict response to preserve referential integrity.

US-09: Restock a Vehicle

As an administrator, I want to increase the quantity of a vehicle so that inventory reflects new stock.

- Only administrators can restock; the vehicle must exist and the restock quantity must be a positive integer.
- Invalid requests leave inventory unchanged; successful restocking persists the updated quantity.

Epic D: Purchases

US-10: Purchase a Vehicle

As a customer, I want to purchase a vehicle so that its inventory quantity is updated.

- The customer must be authenticated, the vehicle must exist, the requested purchase quantity must be positive, and sufficient stock must be available.
- A successful purchase reduces stock by the purchased quantity and persists a purchase record atomically (both succeed or fail together).
- Concurrent requests cannot oversell available stock. (MVP note: A purchase is an inventory transaction recorded by the application, not a real payment or legally binding sale.)

US-11: View Purchase History

As a customer, I want to see my previous purchases so that I can review my activity.

- Each purchase record identifies the customer and vehicle and includes quantity, unit purchase price, total price, and timestamp.
- Customers can view only their own records, and historical prices remain unchanged even if the vehicle's current listing price changes later.

7. Core Features: MVP Versus Later

Prioritization rule: A deferred feature will never be started while a required MVP feature is broken, untested, or missing.

7.1 MVP — Required for the Initial Release (P0)

Feature	Priority	Reason
Customer registration, login, and JWT authentication	P0	Explicit assessment requirement
Persistent SQLite database	P0	Explicit assessment requirement (in-memory rejected)
Vehicle creation, listing, and updates	P0	Required core inventory and customer operations
Vehicle search and filtering (make, model, category, price)	P0	Explicit assessment requirement
Admin-only vehicle deletion and restocking	P0	Explicit assessment requirement
Purchase endpoint + persistent purchase history table/view	P0	Explicit assessment requirement + agreed product scope
React SPA with responsive customer & admin interfaces	P0	Explicit assessment requirement
Automated backend tests & TDD commit history	P0	Core assessment evaluation criteria
README.md (with "My AI Usage"), PROMPTS.md, Test Report, Screenshots	P0	Mandatory submission deliverables

7.2 Later — Only After the MVP Is Reliable (P1–P3)

Feature	Priority	Reason for Deferral
Live application deployment (Vercel / Render / Railway)	P1	Optional assessment bonus; deferred until local MVP is verified

Feature	Priority	Reason for Deferral
Pagination, advanced sorting, and vehicle image uploads	P1	Enhances presentation but adds asset/query complexity
Admin inventory analytics, CSV export, and audit logs	P2	Useful enhancement, not a core assessment requirement
Soft deletion, email verification, and refresh tokens	P2	Expands scope beyond MVP requirements
Real payment integration, multi-branch support, AI recommendations	P3	Unnecessary external risk and scope creep

8. Functional Requirements

8.1 Vehicle Data Model

Prices must use a consistent decimal representation (avoiding binary floating-point rounding errors in monetary calculations).

Field	Type	Constraint
`id`	Integer or UUID	Unique primary key, generated by the system
`make`	String	Required, non-empty
`model`	String	Required, non-empty
`category`	String	Required, non-empty (e.g., Sedan, SUV, Truck, EV)
`price`	Decimal	Required, strictly greater than zero (> 0)
`quantity`	Integer	Required, zero or greater (≥ 0)
`created_at` / `updated_at`	Timestamp	Generated and updated automatically by the system

8.2 Purchase Data Model

Field	Purpose
`id`	Unique purchase transaction identifier
`user_id`	Foreign key identifying the purchasing customer
`vehicle_id`	Foreign key identifying the purchased vehicle
`quantity`	Number of units purchased (positive integer)

Field	Purpose
`unit_price`	Snapshot of agreed vehicle price at the time of purchase
`total_price`	Total recorded transaction value (`quantity * unit_price`)
`created_at`	UTC timestamp of the purchase transaction

8.3 REST API Endpoints & Status Conventions

Method	Endpoint	Purpose	Access
`POST`	`/api/auth/register`	Register a new customer account	Public
`POST`	`/api/auth/login`	Authenticate user and issue JWT	Public
`GET`	`/api/vehicles`	List all available vehicles	Authenticated
`GET`	`/api/vehicles/search`	Search and filter vehicles by make, model, category, price	Authenticated
`POST`	`/api/vehicles`	Add a new vehicle to inventory	Admin only
`PUT`	`/api/vehicles/{id}`	Update an existing vehicle's details	Admin only
`DELETE`	`/api/vehicles/{id}`	Delete a vehicle listing	Admin only
`POST`	`/api/vehicles/{id}/purchase`	Purchase vehicle stock and create purchase record	Authenticated
`POST`	`/api/vehicles/{id}/restock`	Increase vehicle stock quantity	Admin only
`GET`	`/api/purchases/me`	View authenticated user's own purchase history	Authenticated

HTTP Status Code Conventions: `200 OK` (read/update), `201 Created` (resource creation), `204 No Content` (deletion), `400 Bad Request` / `422 Unprocessable Content` (validation failures), `401 Unauthorized` (missing/invalid token), `403 Forbidden` (insufficient role permissions), `404 Not Found` (nonexistent resource), and `409 Conflict` (insufficient stock, duplicate email, or referential integrity conflict on delete).

9. Non-Functional Requirements

- **Security:** Passwords hashed with bcrypt/passlib; JWT secrets loaded via environment variables and excluded from Git; server-side role checks; strict isolation of customer purchase history; CORS restricted to intended frontend origins; zero privilege escalation via public registration.

- **Reliability & Data Integrity:** Persistent SQLite database file; atomic transactions for purchase creation and stock decrementing; database constraints preventing negative stock; protection against concurrent overselling.
- **Maintainability:** Clean separation between API routing, service-layer business rules, and database persistence without over-engineered enterprise bloat.
- **Usability & Accessibility:** Responsive layout for desktop and mobile; clear input labels and focus states; disabled purchase buttons when stock reaches zero; visible loading, success, and error banners.
- **Performance Targets:** Typical local API reads under 500 ms for test datasets and immediate UI state feedback during user interactions.

10. TDD and Testing Strategy

Testing follows a strict test-first **Red-Green-Refactor** workflow reflected in the Git commit history:

1. **RED:** Write an automated test describing the expected behaviour and run it to confirm it fails for the expected reason.
2. **GREEN:** Write the smallest reasonable implementation to make the test pass.
3. **REFACTOR:** Clean up duplication or improve clarity while keeping all tests green.

Testing Layers & High-Priority Scenarios

- **Authentication & Authorization:** Successful registration, duplicate email rejection, invalid email/password validation, valid/invalid login, missing/expired JWT rejection, and 403 Forbidden enforcement when a customer attempts admin routes.
- **Vehicle Management & Search:** Valid vehicle creation, negative/zero price rejection, negative/fractional stock rejection, updating nonexistent vehicles, preventing deletion of vehicles with purchase history, and combined search filters (make, model, category, min/max price, reversed price bounds).
- **Purchasing & Restocking:** Valid purchase stock reduction, insufficient stock rejection (zero stock), negative/zero purchase quantity rejection, atomic rollback on failure, concurrent attempts to buy the final unit, and admin restock validation.
- **Coverage Goal:** Target ≥ 90% line coverage across core backend business logic with 100% of critical authorization and inventory rules verified.

11. Success Metrics

Category	Metric	Target
Functional	Required API endpoints & MVP user journeys implemented	100% working and verified
Security	Unauthorized admin operations rejected	100% of tested cases return 401/403

Category	Metric	Target
Integrity	Negative inventory or invalid purchase modifying stock	0 successful cases
Testing	Core backend business rules tested & line coverage	100% critical rules covered; target ≥ 90% coverage
Process	TDD Git history & AI co-author attribution	Authentic Red-Green-Refactor commits with trailers
Deliverables	README.md, PROMPTS.md, Test Report, Screenshots	100% complete, truthful, and reproducible

12. Edge Cases and Product Decisions

Scenario	Expected System Behaviour
Inventory is empty / Search has no matches	Return empty list (`200 OK`) and display a clear empty/no-results UI state
Vehicle stock is zero	Disable Purchase button in React UI and reject backend purchase requests (`409 Conflict`)
Customer attempts to add, update, restock, or delete vehicle	Backend rejects request with `403 Forbidden`
Duplicate registration email	Reject registration with `409 Conflict` without creating duplicate account
Negative/zero price or negative quantity	Reject request at validation boundary (`422 Unprocessable Content` / `400 Bad Request`)
Purchase quantity exceeds available stock	Reject purchase with `409 Conflict` and leave inventory unchanged
Minimum price filter exceeds maximum price filter	Return validation error (`400` / `422`) and show helpful message
Two customers purchase the final unit simultaneously	Atomic transaction ensures at most one purchase succeeds; the second receives `409 Conflict`
Admin updates a vehicle's price after purchases exist	Existing purchase history records retain their original `unit_price` snapshot
Admin attempts to delete a vehicle with purchase history	Reject deletion with `409 Conflict` to preserve historical purchase records

13. Cut-Off Scope & Priority Order

Explicitly Excluded from MVP: Real payment gateways, credit card storage, financing/loans, refunds, invoicing/tax calculations, shipping/delivery, multi-branch support, supplier procurement, microservices, Kubernetes, distributed caching, AI recommendations, and email notifications.

Strict Cut-Off Priority Order (if time becomes constrained):

1. Correctness and required functionality.
2. Authentication and authorization.
3. Inventory integrity and atomic purchasing.
4. Meaningful automated tests (Red-Green-Refactor).
5. Clear architecture and maintainability.
6. Required documentation and AI disclosure (`README.md`, `PROMPTS.md`, test report).
7. Essential frontend usability.
8. Visual polish.
9. Optional cloud deployment and enhancements.

14. UX Requirements and Visual Direction

CarCraft uses a **Hybrid UX Direction**: a polished, trustworthy customer showroom paired with an efficient, high-clarity administrative dashboard.

- **Customer Experience:** Clean navigation bar, instant search bar, category and price filter controls, responsive vehicle cards displaying clear pricing and stock badges ("In Stock: X" vs. "Out of Stock"), disabled purchase button when stock is 0, purchase history drawer/page, and non-intrusive loading/error states.
- **Admin Experience:** Structured inventory management table/grid, quick modal or inline forms to add/edit vehicles, one-click restock input, confirmation prompt before deleting listings, and clear visual indicators for low or zero stock.

15. Technical Architecture

The repository follows a clean monorepo layout separating backend API, business logic, database persistence, and the React frontend:

```
CarCraft/
├── backend/
│   ├── app/
│   │   ├── main.py          # FastAPI app initialization & CORS
│   │   ├── database.py      # SQLite connection & SQLAlchemy session setup
│   │   ├── models/          # SQLAlchemy ORM models (User, Vehicle, Purchase)
│   │   ├── schemas/         # Pydantic validation schemas
│   │   ├── auth/            # Password hashing, JWT creation & role dependencies
│   │   ├── services/        # Business rules (inventory, atomic purchase, restock)
│   │   └── api/             # FastAPI route controllers
│   ├── tests/               # pytest test suite (auth, vehicles, purchases, concurrency)
```

```
│   ├── requirements.txt
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── components/      # Navbar, VehicleCard, SearchFilterBar, AdminModals
│   │   ├── pages/           # Login, Register, Dashboard, PurchaseHistory, AdminPanel
│   │   ├── services/        # API client helper
│   │   └── App.jsx
│   └── package.json
├── screenshots/             # Verified application screenshots
├── test-report/             # Actual pytest & frontend test execution reports
├── README.md                # Setup instructions, architecture & "My AI Usage"
├── PROMPTS.md               # Raw, unedited AI conversation logs / links
└── .gitignore
```

16. AI Usage and Authenticity Requirements

- **Engineering Ownership:** AI is used as a tutor, pair-programming reviewer, and test brainstormer—never as an unverified code generator. Every line of code must be understood and explainable in simple English during the interview.
- **Mandatory Git Co-Author Trailer:** Every commit involving AI assistance must end with two empty lines followed by:
  `Co-authored-by: AI Tool Name <AI@users.noreply.github.com>`
- **Mandatory README Section ("My AI Usage"):** Details which AI tools were used, how they assisted (brainstorming, boilerplate, debugging, test cases), what suggestions were modified or rejected, and personal reflection on workflow impact.
- **Mandatory Root `PROMPTS.md`:** Preserves raw, unedited AI chat logs or public share links without AI-generated summaries.

17. Risks and Mitigations

Risk	Impact	Mitigation Strategy
Tight deadline (25 Oct, 10:00 PM IST)	Incomplete submission	Strictly enforce MVP scope and follow the 9-step priority order
Beginner Git experience	Messy or missing commit history	Run guided, step-by-step Git commits after each Red and Green phase
AI-generated code not understood	Poor interview performance	Verify every file with a plain-English self-check before committing
Purchase race conditions	Oversold / negative stock	Use atomic database updates and write explicit stock boundary tests
Missing AI logs at deadline	Disqualification on deliverables	Update `PROMPTS.md` and co-author trailers continuously as we build

18. Definition of Done & 19. Final Decision Summary

A feature is done only when: (1) the requirement is understood, (2) a failing test was written first (Red), (3) the minimal implementation passes the test (Green), (4) all existing tests pass, (5) edge cases and role permissions are verified, (6) code is refactored for clarity, and (7) the change is committed to Git with accurate co-author attribution.

Decision Area	Agreed Direction
Primary Users & Roles	Customers (browse, filter, purchase, view history) & Admins (CRUD, restock)
Technical Stack	Python + FastAPI + SQLite + SQLAlchemy (Backend) | React + Tailwind CSS + Vite (Frontend)
Core Priorities	1. Strict TDD | 2. Security & Auth | 3. Edge-Case Testing | 4. Clean Code | 5. Authentic AI Logs